

Flutter - Navigation and Routing

Flutter stellt zwei Kernkonzepte für die Navigation in einer App zur Verfügung: `Navigator` und `Router`. Beide Konzepte ermöglichen es, zwischen verschiedenen Bildschirmen zu navigieren.

Navigator

Die Klasse `Navigator` verwaltet eine Stack von `Route`-Objekten und bietet Methoden zum Verwalten des Stacks. Ein `Route`-Objekt repräsentiert eine Bildschirmseite, die auf dem Bildschirm angezeigt wird. Sie kümmert sich auch um die Animationen, die beim Wechseln von einer Seite zur anderen stattfinden.

Um zu einer Seite zu navigieren, greift man auf den `Navigator` über den `BuildContext` zu und ruft die Methode `push` auf. Um zur vorherigen Seite zurückzukehren, ruft man die Methode `pop` auf.

```
child: const Text('Open second screen'),
onPressed: () {
  Navigator.of(context).push(
    MaterialPageRoute(builder: (context) => const SecondScreen()),
  );
},
```

Weitere Infos und Beispiele findet man hier: [Navigation](#)

Named Routes

HINWEIS: Die Flutter-Dokumentation empfiehlt tatsächlich **NICHT** die Verwendung von Named Routes, wenn die App nur wenige Seiten hat. In diesem Fall ist es einfacher, den `Navigator` zu verwenden.

Named Routes sind eine Möglichkeit, die Navigation in Flutter zu organisieren. Jede benannte Route ist mit einem Namen verknüpft, der in der App verwendet wird, um auf die Route zu verweisen.

Die Routen werden in der `MaterialApp` definiert:

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const FirstScreen(),  
    '/second': (context) => const SecondScreen(),  
  },  
);
```

Um zu einer benannten Route zu navigieren, verwendet man den `Navigator.pushNamed`-Befehl:

```
child: const Text('Open second screen'),  
onPressed: () {  
  Navigator.pushNamed(context, '/second');  
},
```

Hinweis in der Doku, warum Named Routes nicht empfohlen werden:

- *Although named routes can handle deep links, the behavior is always the same and can't be customized. When a new deep link is received by the platform, Flutter pushes a new Route onto the Navigator regardless of where the user currently is.*
- *Flutter also doesn't support the browser forward button for applications using named routes. For these reasons, we don't recommend using named routes in most applications.*

Navigator-Beispiel

main.dart

```
void main() {  
  runApp(const MyApp());  
}  
  
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(title: 'Flutter Routing Demo', home: const  
      ↪ MyHomePage());  
  }  
}
```

my_home_page.dart

```
class MyHomePage extends StatelessWidget {  
  const MyHomePage({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Flutter Routing Demo')),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            Navigator.push(  
              context,  
              MaterialPageRoute(builder: (context) => const  
                ↪ SecondRoute()),  
            );  
          }  
        );  
      );  
    );  
  }  
}
```

```
        },
        child: const Text('Open route'),
      ),
    ),
  );
}
```

second_route.dart

```
class SecondRoute extends StatelessWidget {
  const SecondRoute({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Second Route')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Go back!'),
        ),
      ),
    );
  }
}
```

Router

Flutter-Anwendungen mit erweiterten Navigations- und Routing-Anforderungen (wie z. B. eine Web-App, die direkte Links zu jedem Bildschirm verwendet, oder eine App mit mehreren Navigator-Widgets) sollten ein Routing-Paket wie **go_router** verwenden, das den Routenpfad analysieren und den Navigator konfigurieren kann, wann immer die App einen neuen *Deep Link* erhält.

go_router

Das Routing-Paket **go_router** bietet eine einfache Möglichkeit, die Navigation in einer Flutter-App zu verwalten. Es ist einfach zu konfigurieren und bietet viele Funktionen, die in der Standard-Navigator-Konfiguration nicht verfügbar sind.

Installiere das Paket von der Kommandozeile:

```
flutter pub add go_router
```

Das vorherige Beispiel mit **go_router** würde so aussehen:

main.dart

```
void main() {  
  runApp(const MyApp());  
}  
  
/// The route configuration.  
final GoRouter _router = GoRouter(  
  routes: <RouteBase>[  
    GoRoute(  
      path: '/',  
      builder: (BuildContext context, GoRouterState state) {  
        return const MyHomePage();  
      }  
    )  
  ],  
);
```

```
    },  
    routes: <RouteBase>[  
      GoRoute(  
        path: 'second',  
        builder: (BuildContext context, GoRouterState state) {  
          return const SecondRoute();  
        },  
      ),  
    ],  
  ),  
],  
);
```

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp.router(  
      title: 'Flutter Routing Demo',  
      routerConfig: _router,  
    );  
  }  
}
```

my_home_page.dart

```
class MyHomePage extends StatelessWidget {  
  const MyHomePage({super.key});  
  
  @override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text('Flutter Routing Demo')),  
    body: Center(  
      child: ElevatedButton(  
        onPressed: () {  
          context.go('/second');  
        },  
        child: const Text('Open route'),  
      ),  
    ),  
  );  
}
```

second_route.dart

```
class SecondRoute extends StatelessWidget {  
  const SecondRoute({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Second Route')),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            context.go('/');  
          },  
          child: const Text('Go back!'),  
        ),  
      ),  
    );  
  }  
}
```

```
    ),  
  );  
}  
}
```

Weitere Informationen und Beispiele findest du hier: [go_router](#)

Verwendung von Tabs

Eine andere Art der Navigation in Flutter ist die Verwendung von Tabs. Dabei bleibt man auf der gleichen Seite, jedoch wird der Inhalt der Seite durch Tabs gewechselt.

Um mit **Tabs** zu arbeiten, sind folgende drei Schritte erforderlich:

1. Einen `TabController` erstellen.
2. Die `TabBar` und `TabBarView`-Widgets verwenden.
3. Die Inhalte für die Tabs erstellen.

TabController

Ein `TabController` wird verwendet, um die Tabs zu steuern. Am einfachsten ist es, den `DefaultTabController` zu verwenden, der den `TabController` automatisch erstellt und verwaltet.

tab_bar_demo.dart

```
class TabBarDemo extends StatelessWidget {  
  const TabBarDemo({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return DefaultTabController(  

```

```
length: 3,
child: Scaffold(
  appBar: AppBar(
    bottom: const TabBar(
      tabs: [
        Tab(icon: Icon(Icons.directions_car)),
        Tab(icon: Icon(Icons.directions_transit)),
        Tab(icon: Icon(Icons.directions_bike)),
      ],
    ),
    title: const Text('Tabs Demo'),
  ),
  body: const TabBarView(
    children: [
      Icon(Icons.directions_car),
      Icon(Icons.directions_transit),
      Icon(Icons.directions_bike),
    ],
  ),
),
);
}
```

main.dart

```
/// The route configuration.
final GoRouter _router = GoRouter(
  routes: <RouteBase>[
    GoRoute(
      path: '/',
```

```
builder: (BuildContext context, GoRouterState state) {  
  return const MyHomePage();  
},  
routes: <RouteBase>[  
  GoRoute(  
    path: 'second',  
    builder: (BuildContext context, GoRouterState state) {  
      return const SecondRoute();  
    },  
  ),  
  GoRoute(  
    path: 'tab_bar',  
    builder: (BuildContext context, GoRouterState state) {  
      return const TabBarDemo();  
    },  
  ),  
],  
),  
],  
);
```

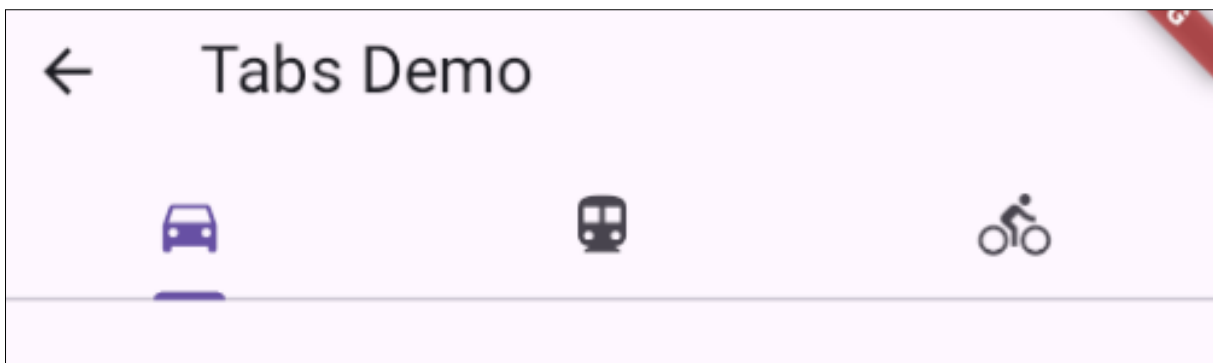
my_home_page.dart

@override

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text('Flutter Routing Demo')),  
    body: Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.min,  
        children: [  

```

```
ElevatedButton(  
  onPressed: () {  
    context.go('/second');  
  },  
  child: const Text('Open route'),  
)  
  ,  
  SizedBox(height: 20),  
  ElevatedButton(  
    onPressed: () {  
      context.go('/tab_bar');  
    },  
    child: const Text('Open Tab Bar Demo'),  
  ),  
,  
)  
,  
)  
);  
}
```



Daten an eine Seite übergeben

Oft möchte man bestimmte Daten an eine Seite übergeben, z.B. der Listen-Eintrag, auf den gedrückt wurde. Wir sehen uns das anhand einer einfachen ToDo-App an. Erstellen wir dazu eine Modell-Klasse Todo.

todo.dart

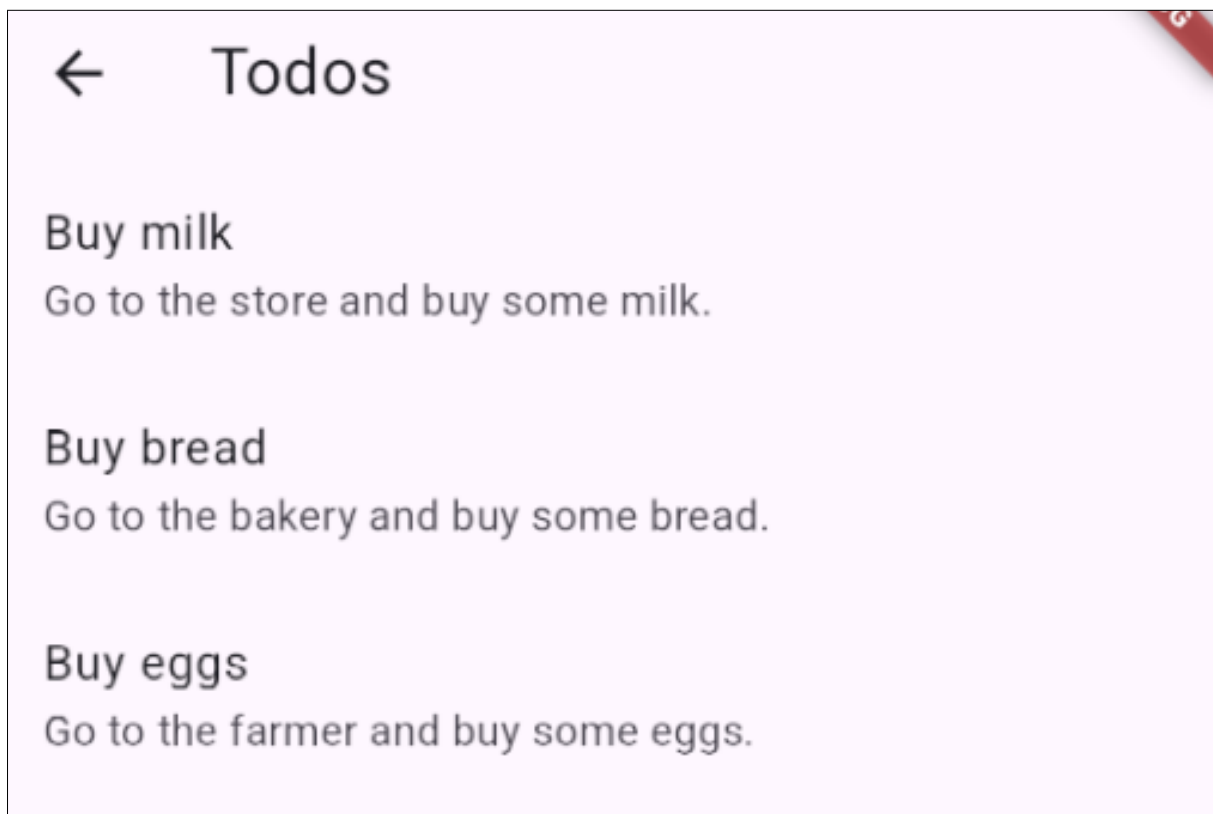
```
class Todo {  
  final String title;  
  final String description;  
  
  const Todo(this.title, this.description);  
}
```

Nun generieren wir ein paar Beispiel-Todos und zeigen sie in einer Liste an.

todos_screen.dart

```
class TodosScreen extends StatelessWidget {  
  final List<Todo> todos = [  
    Todo('Buy milk', 'Go to the store and buy some milk.'),  
    Todo('Buy bread', 'Go to the bakery and buy some bread.'),  
    Todo('Buy eggs', 'Go to the farmer and buy some eggs.'),  
  ];  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Todos')),  
      //passing in the ListView.builder  
      body: ListView.builder(  
        itemCount: todos.length,
```

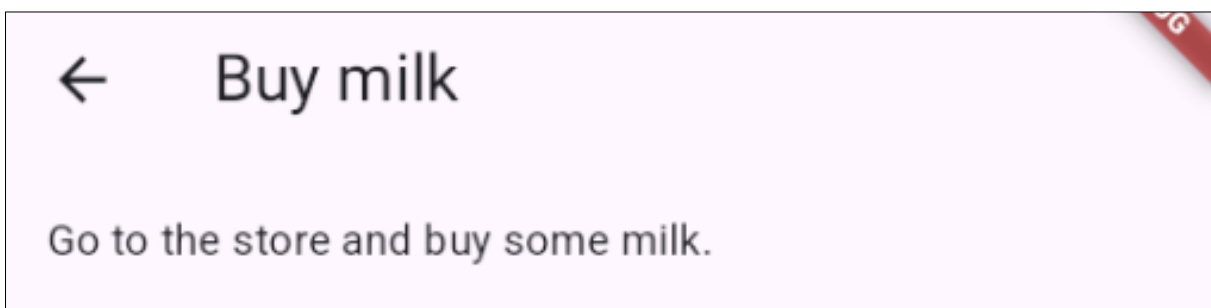
```
    itemBuilder: (context, index) {  
      return ListTile(  
        title: Text(todos[index].title),  
        subtitle: Text(todos[index].description),  
      );  
    },  
  ),  
);  
}
```



Wenn auf einen ToDo-Eintrag gedrückt wurde, soll die `TodoDetail`-Seite geöffnet, die das ausgewählte ToDo anzeigt.

`todo_detail.dart`

```
class TodoDetail extends StatelessWidget {  
  final Todo todo;  
  
  const TodoDetail({required this.todo, super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text(todo.title)),  
      body: Padding(  
        padding: const EdgeInsets.all(16.0),  
        child: Text(todo.description),  
      ),  
    );  
  }  
}
```



Wie man sieht, wird das ausgewählte Todo als Konstruktor-Parameter an die `TodoDetail`-Seite übergeben.

Nun müssen wir noch in der `TodosScreen`-Seite den Navigator verwenden, um die `TodoDetail`-Seite zu öffnen:

todos_screen.dart

@override

```
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text('Todos')),  
    //passing in the ListView.builder  
    body: ListView.builder(  
      itemCount: todos.length,  
      itemBuilder: (context, index) {  
        return ListTile(  
          title: Text(todos[index].title),  
          subtitle: Text(todos[index].description),  
          onTap: () { // <--  
            Navigator.push(  
              context,  
              MaterialPageRoute(  
                builder: (context) => TodoDetail(todo: todos[index]),  
              ),  
            );  
          },  
        );  
      },  
    ),  
  );  
}
```

Jetzt können wir eine neue Route für die Todos-Seite hinzufügen und per Button-Klick in der MyHomePage-Seite öffnen:

main.dart

```
routes: <RouteBase>[  
  GoRoute(  
    path: 'second',
```

```
        builder: (BuildContext context, GoRouterState state) {
          return const SecondRoute();
        },
      ),
      GoRoute(
        path: 'tab_bar',
        builder: (BuildContext context, GoRouterState state) {
          return const TabBarDemo();
        },
      ),
      GoRoute( // <--
        path: 'todos',
        builder: (BuildContext context, GoRouterState state) {
          return const TodosScreen();
        },
      ),
    ],
```

my_home_page.dart

```
...
  SizedBox(height: 20),
  ElevatedButton(
    onPressed: () {
      context.go('/todos');
    },
    child: const Text('Open Todos Demo'),
  ),
  ...
```

Variante 2: RouteSettings

Eine andere Möglichkeit, Daten an eine Seite zu übergeben, ist die Verwendung von RouteSettings. Dabei wird der RouteSettings-Klasse ein arguments-Parameter übergeben, der die Daten enthält.

```
class TodosScreen extends StatelessWidget {  
  final List<Todo> todos = const [  
    Todo('Buy milk', 'Go to the store and buy some milk.'),  
    Todo('Buy bread', 'Go to the bakery and buy some bread.'),  
    Todo('Buy eggs', 'Go to the farmer and buy some eggs.'),  
  ];  
  
  const TodosScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Todos')),  
      body: ListView.builder(  
        itemCount: todos.length,  
        itemBuilder: (context, index) {  
          return ListTile(  
            title: Text(todos[index].title),  
            subtitle: Text(todos[index].description),  
            onTap: () {  
              // <--  
              Navigator.push(  
                context,  
                MaterialPageRoute(  
                  builder: (context) => TodoDetail(), // <--  
                ),  
              );  
            },  
          );  
        },  
      ),  
    );  
  }  
}
```

```
        settings: RouteSettings(arguments: todos[index]),
    ↪ // <--
    ),
  );
},
);
},
),
);
}
}
```

Mit der `ModalRoute.of()`-Methode kann man dann auf die `RouteSettings` zugreifen und die Daten auslesen:

```
class TodoDetail extends StatelessWidget {
  final Todo todo;

  const TodoDetail({super.key}); // <--

  @override
  Widget build(BuildContext context) {
    final todo = ModalRoute.of(context)!.settings.arguments as Todo;
    ↪ // <--

    return Scaffold(
      appBar: AppBar(title: Text(todo.title)),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Text(todo.description),
      ),
    ),
  ),
}
```

```
    );  
  }  
}
```

Daten an Screen zurück geben

Manchmal möchte Daten an den vorherigen Screen zurückgeben, z.B. wenn eine Auswahl vorzunehmen ist und das Ergebnis an den vorherigen Screen übergeben werden soll, um damit weiter arbeiten zu können.

Dazu wird die `Navigator.pop()`-Methode verwendet, die einen optionalen `result`-Parameter übergeben bekommt.

HomeScreen

```
class HomeScreen extends StatelessWidget {  
  const HomeScreen({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Returning Data Demo')),  
      body: const Center(child: SelectionButton()),  
    );  
  }  
}
```

Das `SelectionButton`-Widget öffnet den `SelectionScreen`-Screen und wartet auf das Ergebnis.

SelectionButton

```
class SelectionButton extends StatefulWidget {
```

```
const SelectionButton({super.key});

@override
State<SelectionButton> createState() => _SelectionButtonState();
}

class _SelectionButtonState extends State<SelectionButton> {
  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        _navigateAndDisplaySelection(context);
      },
      child: const Text('Pick an option, any option!'),
    );
  }

  Future<void> _navigateAndDisplaySelection(BuildContext context)
  ↪ async {
    final result = await Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const
  ↪ SelectionScreen()),
    );
  }
}
```

Im SelectionScreen-Screen wird die Auswahl getroffen und an den vorherigen Screen zurückgegeben.

SelectionScreen

```
class SelectionScreen extends StatelessWidget {
  const SelectionScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Pick an option')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Padding(
              padding: const EdgeInsets.all(8),
              child: ElevatedButton(
                onPressed: () {
                  Navigator.pop(context, 'Yep!'); // <--
                },
                child: const Text('Yep!'),
              ),
            ),
            Padding(
              padding: const EdgeInsets.all(8),
              child: ElevatedButton(
                onPressed: () {
                  Navigator.pop(context, 'Nope. '); // <--
                },
                child: const Text('Nope. '),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

```
        ),  
      ),  
    );  
  }  
}
```

Finalisieren wir nun die `SelectionButton`-Klasse, um das Ergebnis anzuzeigen.

SelectionButton

```
...  
Future<void> _navigateAndDisplaySelection(BuildContext context) async  
→ {  
  final result = await Navigator.push(  
    context,  
    MaterialPageRoute(builder: (context) => const SelectionScreen()),  
  );  
  
  if (!context.mounted) return;  
  
  ScaffoldMessenger.of(context)  
    ..removeCurrentSnackBar()  
    ..showSnackBar(SnackBar(content: Text('$result')));  
}  
...
```